

Pulse-Oximeter: Modulararbeit für Mobile und Ubiquitous Computing

Dokumentation der Modulararbeit: 2024, Juli 2024

Maria Lyoteva, Tsvetan Stanchev
Studiengang: Künstliche Intelligenz
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Deutschland

Zusammenfassung—Pulse-Oximeter ist eine Android-Anwendung, die den Benutzern die Möglichkeit bietet, ihren Puls und ihre Sauerstoffsättigungswerte zu überwachen. Der Prozess beginnt mit einem ESP32-Mikrocontroller, der mit einem MAX30100-Sensor ausgestattet ist. Dieser sammelt Daten und sendet sie über USB an einen PC oder Laptop mit Node-RED. Node-RED ist für den Empfang, die Aggregation und die Visualisierung dieser Sensordaten zuständig. Die Daten werden dann an einen MQTT-Broker übertragen, der lokal gehostet wird. Schließlich abonniert eine Smartphone-App den MQTT-Broker, um die Daten zu empfangen, sie zu visualisieren und lokal mit SQLite zu speichern.

Index Terms—ESP32, MAX30100, Node-Red, MQTT, Android.

I. PROJEKTBSCHREIBUNG

A. Einführung

Im Rahmen der Projektarbeit „Mobile & Ubiquitous Computing“ wird eine Biosensor-App entwickelt, die Echtzeitdaten eines Herzfrequenz- und Pulsoximetersensors (MAX30100) erfasst und visualisiert. Die App nutzt die Möglichkeiten des IoT („Internet of Things“), um Sensordaten zu aggregieren, zu visualisieren und für die spätere Analyse zu speichern.

B. Ziele des Projekts

Die primären Ziele dieses Projekts sind:

- **Sensor Data Acquisition:** Echtzeitdaten eines optischen Herzfrequenz- und Pulsoximetersensors (MAX30100) mit einem ESP32-Mikrocontroller erfassen.
- **Data Aggregation and Visualization:** Aggregation und Visualisierung der Sensordaten mit Node-RED und Veröffentlichung über das MQTT-Protokoll.
- **Mobile Application Development:** Entwicklung einer Android-Anwendung zur Anzeige und Speicherung der Sensordaten.

C. Architektur des Projekts

Die Architektur des Projekts umfasst vier Hauptkomponenten:

- **ESP32 Microcontroller mit MAX30100 Sensor:**
 - Der MAX30100 Sensor ist über die I2C-Schnittstelle mit dem ESP32 Mikrocontroller verbunden, welcher die Herzfrequenz und die Blutsauerstoffsättigung erfasst

und die Daten über eine serielle USB-Verbindung an einen PC oder Laptop sendet.

- **PC/Laptop mit Node-RED:**

- Node-RED liest die Sensordaten vom ESP32, verarbeitet und visualisiert sie. Die Daten werden anschließend über das MQTT-Protokoll veröffentlicht.

- **MQTT Broker:**

- Der MQTT-Broker (lokal oder cloudbasiert) dient als Vermittler für die Datenkommunikation zwischen Node-RED und der mobilen Anwendung.

- **Smartphone Anwendung:**

- Die mobile Anwendung empfängt die Sensordaten vom MQTT-Broker, zeigt sie in einem benutzerfreundlichen Dashboard an und speichert sie lokal in einer SQLite-Datenbank.

II. FUNKTIONSWEISE DES SENSORS

A. Einführung in den MAX30100-Sensor

Der MAX30100 ist ein integrierter Pulsoximeter- und Herzfrequenzsensor, der für tragbare Gesundheitsanwendungen entwickelt wurde. Der Sensor misst die Sauerstoffsättigung im Blut (SpO₂) und die Herzfrequenz (HR) durch die Analyse der Lichtabsorption bei verschiedenen Wellenlängen.

B. Funktionsprinzip

Der MAX30100-Sensor arbeitet, indem er Licht durch das Gewebe sendet und die Lichtintensität misst, die auf der anderen Seite empfangen wird. Zwei LEDs, eine rote und eine infrarote, werden abwechselnd eingeschaltet, und ein Photodetektor misst das durch den Finger hindurchgelassene Licht. Durch die Analyse der Lichtabsorption bei den beiden verschiedenen Wellenlängen kann der Sensor die SpO₂ und die HR bestimmen.

C. Sensorbetrieb

- **LEDs und Photodetektor:** Die rote LED und die infrarote LED senden Licht durch das Gewebe, und der Photodetektor misst das durchgelassene Licht.
- **Datenanalyse:** Der Sensor analysiert die Lichtabsorption, um die SpO₂ und die HR zu bestimmen.

- **Datenübertragung:** Die erfassten Daten werden über die I2C-Schnittstelle an den ESP32 Mikrocontroller übertragen.

D. Vorteile und Einschränkungen

- **Vorteile:** Hohe Genauigkeit, Echtzeitmessungen, geringe Größe und Energieverbrauch.
- **Einschränkungen:** Empfindlichkeit gegenüber Bewegungsartefakten, Lichtinterferenzen und Hautpigmentierungen.

III. PROJEKTPLANUNG UND VORGEHEN

A. Teamaufteilung

Die Teamaufteilung erfolgte wie folgt:

Aufgabe	Verantwortlich
Zusammenbau der Hardware	Tsvetan
Arduino Code	Tsvetan
Vorbereitung des Mosquitto Brokers	Maria
Erstellung des Node-Red-Flows	Tsvetan und Maria
Frontend	Maria
Backend	Maria: MainActivity, MqttData, MqttManager, SoundManager Tsvetan: OptionsActivity, DataTableActivity, DataAdapter, DatabaseHelper, ChartManager
Dokumentation	Tsvetan und Maria

Tabelle I
PROJEKTAUFGABEN UND VERANTWORTLICHKEITEN

B. Zeitleiste

Ein detaillierter Zeitplan, der die wichtigsten Meilensteine und Aufgaben darstellt:

Woche	Aufgabe
1-2	Hardware-Setup und Sensorintegration
3-4	Node-RED Konfiguration und Datenvisualisierung
5-6	Entwicklung der Smartphone-App
7-8	Testing, Debugging und Dokumentation

Tabelle II
ZEITPLAN

IV. PROJEKTIMPLEMENTIERUNG

A. Hardware-Konfiguration

1) **Hardware-Komponenten:** Wir haben die folgenden Komponenten verwendet:

- **ESP32 Mikrocontroller:** Ein leistungsfähiger Mikrocontroller mit WLAN- und Bluetooth-Fähigkeiten.
- **MAX30100 Sensor:** Ein integrierter Pulsoximeter- und Herzfrequenzsensor.
- **Breadboard:** Ein Steckbrett zur Herstellung von provisorischen Schaltungen ohne Lötten.
- **Verbindungskabel:** Verschiedene farbige Jumper-Kabel zur Verbindung der Komponenten.

2) **Verkabelung:** Die Konfiguration der Verbindungen wurde wie folgt vorgenommen:

- **VIN** des Sensors wurde mit dem **3.3V** Pin des ESP32 verbunden (braune Leitung). Dies versorgt den Sensor mit der notwendigen Betriebsspannung.
- **GND** des Sensors wurde mit dem **GND** Pin des ESP32 verbunden (rote Leitung). Dies stellt die gemeinsame Masseverbindung her.
- **SCL** des Sensors wurde mit dem **Port 33** Pin des ESP32 verbunden (orange Leitung). Dies ist die serielle Taktleitung für die I2C-Kommunikation.
- **SDA** des Sensors wurde mit dem **Port 32** Pin des ESP32 verbunden (gelbe Leitung). Dies ist die serielle Datenleitung für die I2C-Kommunikation.

Abbildung 1 zeigt ein Beispiel der Hardware-Konfiguration auf einem Breadboard, welches die oben beschriebenen Verbindungen veranschaulicht:

B. Verarbeitung der Sensor-Daten

1) **Programmierung des ESP32 mit der Arduino IDE:** Der ESP32 Mikrocontroller wird mit der Arduino IDE programmiert, um die Herzfrequenz- (BPM) und Sauerstoffsättigungsdaten (SpO2) zu erfassen.

2) **Einbindung der Bibliotheken:** Es werden die **Wire-** und **MAX30100_PulseOximeter-**Bibliotheken eingebunden, die für die Kommunikation mit dem MAX30100-Sensor erforderlich sind.

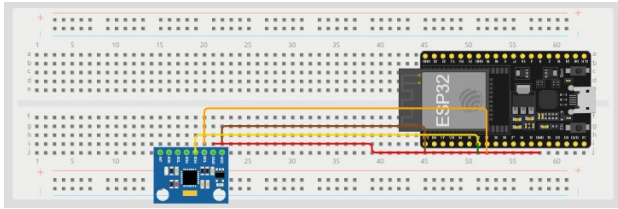


Abbildung 1. Hardware-Konfiguration des ESP32 und MAX30100 Sensors auf einem Breadboard

```
1 #include <Wire.h>
2 #include "MAX30100_PulseOximeter.h"
```

3) *Initialisierung des Sensors*: Der Sensor wird über die I2C-Schnittstelle (SDA zu GPIO 33, SCL zu GPIO 32) initialisiert. Falls die Initialisierung fehlschlägt, bleibt der ESP32 in einer Endlosschleife hängen.

```
1 void setup()
2 {
3   Serial.begin(9600);
4   Serial.println("Initializing pulse oximeter..");
5
6   Wire.begin(32, 33); // Initialize I2C with custom pins if
7                       // necessary (SDA to GPIO 33, SCL to GPIO 32)
8
9   if (!pox.begin()) {
10     Serial.println("FAILED to initialize pulse oximeter");
11     while (1); // Hang indefinitely if initialization fails
12   } else {
13     Serial.println("Pulse oximeter initialized successfully");
14   }
15
16   pox.setOnBeatDetectedCallback(onBeatDetected);
17 }
```

4) *Herzschlag-Erkennung*: Wenn ein Herzschlag erkannt wird, wird das pulseDetected-Flag gesetzt.

```
1 void onBeatDetected() {
2   pulseDetected = true; // Set flag when a beat is detected
3 }
```

5) *Datenverarbeitung*: Die Herzfrequenz- und SpO2-Werte werden über einen Zeitraum von 10 Sekunden gemessen und akkumuliert. Anschließend wird der Durchschnitt berechnet und über die serielle Schnittstelle ausgegeben.

```
1 void loop() {
2
3   pox.update();
4
5   if (pulseDetected) {
6
7     if (startTime == 0) {
8       startTime = millis(); // Start timing
9     }
10
11     elapsedTime = millis() - startTime;
12
13     if (elapsedTime < 10000) { // 10 seconds in milliseconds
14       // Accumulate heart rate and SpO2
```

```
15     heartRateSum += pox.getHeartRate();
16     spo2Sum += pox.getSpO2();
17     measurementsNumber += 1;
18   } else {
19     Serial.print(heartRateSum / measurementsNumber); // Average
20     Serial.print(", ");
21     Serial.println(spo2Sum / measurementsNumber); // Average SpO2
22     // over 10 seconds
23
24     // Reset variables for the next 10-second interval
25     startTime = 0;
26     elapsedTime = 0;
27     heartRateSum = 0;
28     spo2Sum = 0;
29     measurementsNumber = 0;
30     pulseDetected = false;
31   }
32 }
```

C. Node-Red Konfiguration und Datenvisualisierung- I Stufe

Unser Node-RED-Flow simuliert die Erfassung von Herzfrequenz- und Sauerstoffsättigungsdaten durch die Verwendung von zufällig generierten Werten. Die generierten Herzfrequenzwerte liegen im Bereich von 60 bis 80 BPM, während die Sauerstoffsättigungswerte im Bereich von 90 bis 98 Prozent liegen. Diese Simulation ermöglicht eine einfache und effektive Testumgebung, ohne dass echte Sensoren benötigt werden. Die generierten Daten werden in Echtzeit über MQTT an das Dashboard gesendet, wo sie visualisiert werden können. In den nächsten Schritten werden wir unseren Flow im Detail erklären.

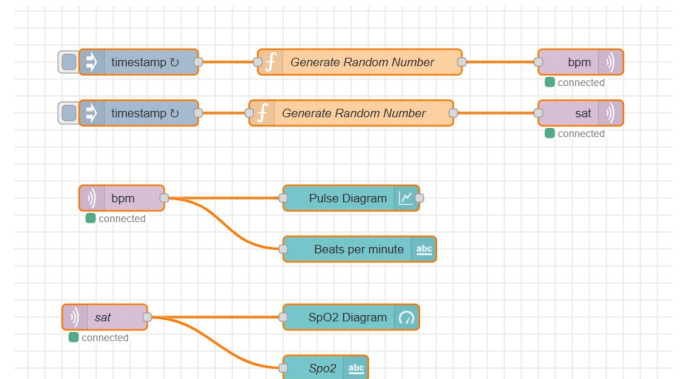


Abbildung 2. Node-RED Flow zur Simulation von Herzfrequenz- und Sauerstoffsättigungsdaten

1) *Injektionsknoten*: Der Flow beginnt mit zwei Injektionsknoten, die in regelmäßigen Abständen Nachrichten auslösen. Diese Knoten senden initiale Nachrichten, die zur Generierung der Zufallsdaten dienen.

- **inject_node** Dieser Knoten sendet jede Sekunde eine Nachricht. Die Nachricht wird an den Funktionsknoten weitergeleitet, der zufällige BPM-Werte generiert.

- **inject_node_2** Dieser Knoten sendet ebenfalls jede Sekunde eine Nachricht. Die Nachricht wird an den zweiten Funktionsknoten weitergeleitet, der zufällige SpO2-Werte generiert.

2) *Funktionsknoten:* Die Funktionsknoten sind so programmiert, dass sie zufällige Werte innerhalb eines bestimmten Bereichs generieren und diese an die entsprechenden MQTT-Ausgabeknoten senden.

- **function_node (Generate Random Number)**
Dieser Knoten generiert eine zufällige Herzfrequenz (BPM) im Bereich von 60 bis 80. Der erzeugte Wert wird dann an den MQTT-Ausgabeknoten gesendet.
- **function_node_2 (Generate Random Number)**
Dieser Knoten generiert eine zufällige Sauerstoffsättigung (SpO2) im Bereich von 90 bis 98. Der erzeugte Wert wird dann an den MQTT-Ausgabeknoten gesendet.

3) *MQTT-Ausgabeknoten:* Die MQTT-Ausgabeknoten senden die generierten Daten an die MQTT-Broker, von wo aus sie an die entsprechenden MQTT-Eingabeknoten weitergeleitet werden.

- **mqtt_out_node**
Sendet die generierten BPM-Daten an das MQTT-Thema "bpm".
- **mqtt_out_node_2**
Sendet die generierten SpO2-Daten an das MQTT-Thema "bat". (Bat steht für B aturation)

4) *MQTT-Eingabeknoten:* Die MQTT-Eingabeknoten empfangen die Daten von den MQTT-Brokern und leiten diese an die Dashboard-Knoten weiter.

- **mqtt_in_node (BPM):**
Empfängt die BPM-Daten vom MQTT-Thema "bpm" und sendet sie an das Diagramm und den Textanzeige-Knoten.
- **mqtt_in_node_2 (SpO2):** Empfängt die SpO2-Daten vom MQTT-Thema "bat" und sendet sie an das Gauge-Diagramm und den Textanzeige-Knoten.

5) *Dashboard-Knoten:* Die Dashboard-Knoten visualisieren die empfangenen Daten. Dies ermöglicht eine einfache Überwachung der simulierten Werte.

- **ui_chart:** Zeigt die BPM-Daten in einem Liniendiagramm an. Dies ermöglicht die visuelle Verfolgung der Herzfrequenz über die Zeit.
- **ui_text:** Zeigt die BPM-Daten als Text an. Dies bietet eine klare und präzise Anzeige der aktuellen Herzfrequenz.
- **ui_text_2:** Zeigt die SpO2-Daten als Text an. Dies bietet eine klare und präzise Anzeige der aktuellen Sauerstoffsättigung.
- **ui_gauge:** Zeigt die SpO2-Daten in einem Gauge-Diagramm an. Dies bietet eine schnelle visuelle Rückmeldung über die Sauerstoffsättigung.

6) *Dashboard Visualisierung und Erklärung:* Sie können unser Dashboard auf dem untenstehenden Bild sehen:

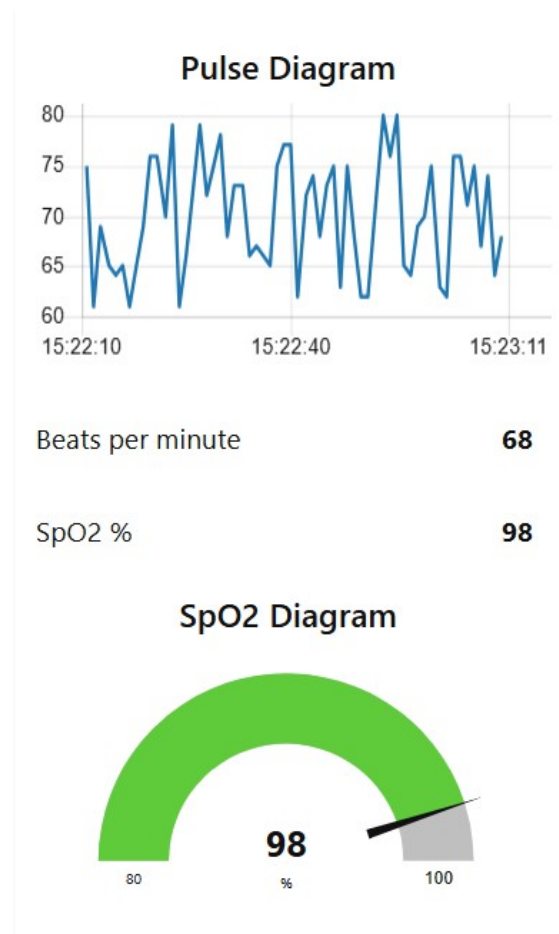


Abbildung 3. Node-Red Dashboard

Das Gauge-Diagramm, das die SpO2-Daten anzeigt, verwendet verschiedene Farben, um unterschiedliche Bereiche der Sauerstoffsättigung im Blut darzustellen:

- **Grün - Normaler Bereich:** 95% bis 100% - Dieser Bereich zeigt eine gute Sauerstoffversorgung des Blutes und der Organe an. (Abbildung 3)
- **Gelb - Leichte Hypoxie:** 90% bis 94% - Werte in diesem Bereich können auf eine leichte Beeinträchtigung der Sauerstoffversorgung hinweisen und sollten überwacht werden. (Abbildung 4)

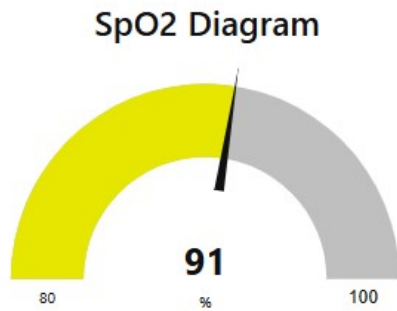


Abbildung 4. Gauge-Diagramm gelb

- **Rot - Schwere Hypoxie** Unter 90% - Werte unter 90% sind besorgniserregend und erfordern sofortige medizinische Intervention. (Abbildung 5)

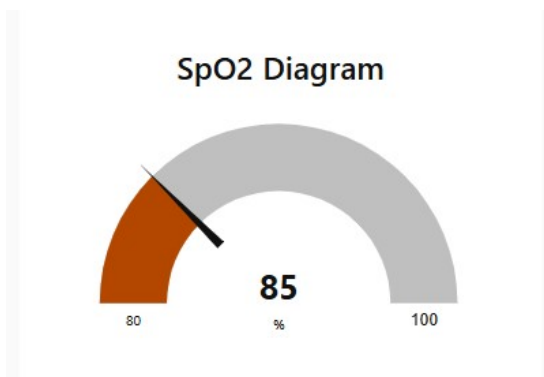


Abbildung 5. Gauge-Diagramm rot

D. Node-Red Konfiguration und Datenvisualisierung- II Stufe

Dieser Node-RED-Flow (Abbildung 6) dient der Verarbeitung und Visualisierung von Herzfrequenz- (BPM) und Sauerstoffsättigungsdaten (SpO2). Daten werden über eine serielle Schnittstelle empfangen, aufgeteilt und dann über MQTT an ein Dashboard zur Visualisierung gesendet.

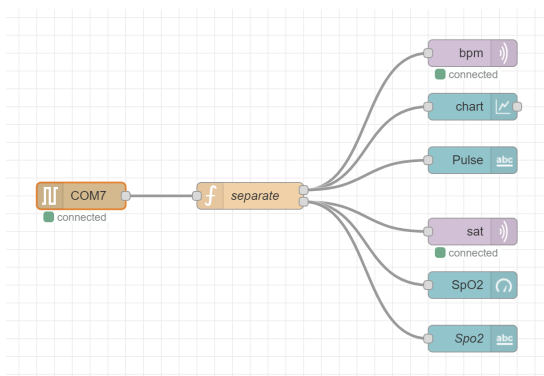


Abbildung 6. Node-Red Flow mit den realen Daten

1) *Injektionsknoten*: Der Flow beginnt mit dem Empfang von Daten über eine serielle Schnittstelle. (COM7 von unserem PC)

2) *Funktionsknoten*: Der Funktionsknoten ist so programmiert, dass sie die empfangenen Daten in zwei getrennte Nachrichten aufteilen, eine für BPM und eine für SpO2.

```
var input = msg.payload.trim(); // Entfernt umgebende Leerzeichen
var values = input.split(","); // Teilt die Eingabe durch Komma
if (values.length === 2) {
  var msg1 = { topic: "topic1", payload: Number(values[0]) };
  var msg2 = { topic: "topic2", payload: Number(values[1]) };
  return [msg1, msg2]; // Sendet beide Nachrichten
} else {
  node.error("Invalid input format");
  return null; // Oder behandelt den Fehler entsprechend
}
```

Abbildung 7. Funktionimplementierung

3) *MQTT- und Dashboard- Knoten*: Die MQTT- und Dashboard-Knoten arbeiten nach dem gleichen Prinzip wie diejenigen im ersten Flow.

E. MQTT Konfiguration

1) *Einleitung*: MQTT (Message Queuing Telemetry Transport) ist ein leichtes Protokoll für die Nachrichtenübermittlung, das häufig in IoT-Anwendungen (Internet of Things) verwendet wird. In unserem Projekt haben wir MQTT lokal installiert und verwenden es zur Kommunikation zwischen verschiedenen Geräten, einschließlich des ESP32-Mikrocontrollers und einer Android-App im Emulator. Zusätzlich nutzen wir MQTXX, um die Datenübertragung zu überprüfen und sicherzustellen, dass die Nachrichten korrekt gesendet und empfangen werden.

2) *Installation und Konfiguration des MQTT-Brokers*: Wir haben einen MQTT-Broker lokal auf unseren PCs installiert, um als zentraler Kommunikationsknotenpunkt zu dienen. Der Broker verwendet den Standardport 1883 und ist so konfiguriert, dass er Nachrichten von verschiedenen MQTT-Clients empfängt und verteilt.

3) *Verwendung von MQTXX zur Überprüfung der Datenübertragung*: MQTXX ist ein MQTT 5.0 Desktop-Client, der uns ermöglicht, die MQTT-Kommunikation zu testen und zu überwachen. Wir verwenden MQTXX, um sicherzustellen, dass die Daten, die vom ESP32 gesendet werden, korrekt empfangen und verarbeitet werden.

Verbindung zum MQTT-Broker herstellen:

- Starten Sie MQTXX und fügen Sie eine neue Verbindung hinzu.

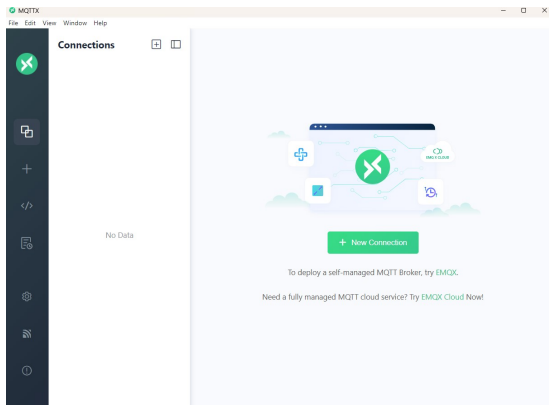


Abbildung 8. MQTTX Homescreen

- Geben Sie die Broker-Adresse (localhost) und den Port (1883) ein.

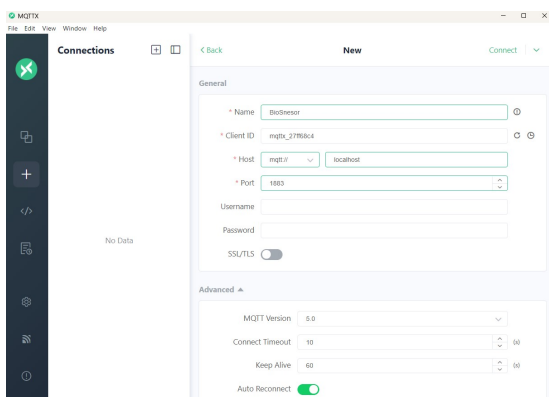


Abbildung 9. neue Verbindung erstellen

- Klicken Sie auf "Connect", um die Verbindung zum lokalen MQTT-Broker herzustellen.

Überprüfen der Datenübertragung:

- Abonnieren Sie die Themen bpm und sat, um die empfangenen Daten anzuzeigen.

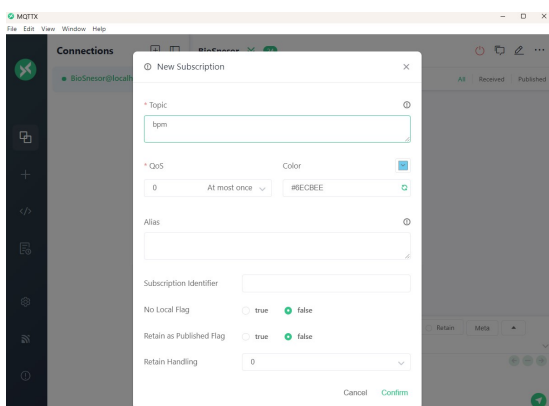


Abbildung 10. Themen (Topics) abonnieren

- Die Verbindung überprüfen.

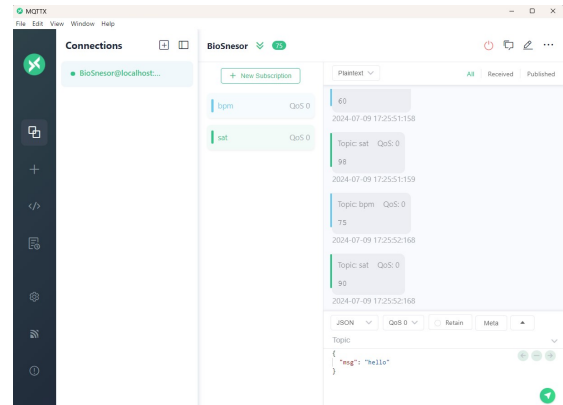


Abbildung 11. Daten visualisieren

F. Entwicklung der Smartphone App

Wir haben uns entschieden, eine Android-App zu entwickeln, die in einem Android Studio Emulator (Pixel 7, API27 Device) läuft. Unser Projekt ist eine Android-Anwendung zur Überwachung und Anzeige von Herzfrequenz- und Sauerstoffsättigungsdaten. Es verwendet MQTT, um Echtzeitdaten zu empfangen, speichert diese Daten in einer lokalen Datenbank und bietet Funktionen zur Ansicht und Verwaltung der Daten über verschiedene UI-Komponenten und Aktivitäten.

Emulator in Android Studio konfigurieren

Um einen Emulator für ein Pixel 7 mit API 27 zu konfigurieren, folgen Sie diesen Schritten:

1) Android Studio öffnen:

Öffnen Sie Android Studio auf Ihrem Computer.

2) AVD Manager öffnen:

- Klicken Sie in der Menüleiste auf Tools.
- Wählen Sie AVD Manager aus dem Dropdown-Menü oder klicken Sie auf das AVD Manager-Symbol in der Symbolleiste.

3) Neues virtuelles Gerät erstellen:

- Klicken Sie im AVD Manager auf Create Virtual Device.
- Wählen Sie Pixel 7 aus der Liste der verfügbaren Geräte.
- Klicken Sie auf Next.

4) Systemabbild auswählen:

- Wählen Sie die Android-Version API 27 aus.
- Wenn das Systemabbild nicht verfügbar ist, klicken Sie auf Download und installieren Sie es.
- Klicken Sie auf Next.

5) AVD-Einstellungen konfigurieren:

- Geben Sie Ihrem virtuellen Gerät einen Namen im Feld AVD Name.
- Überprüfen Sie die restlichen Einstellungen und passen Sie sie nach Bedarf an.
- Klicken Sie auf Finish, um das virtuelle Gerät zu erstellen.

6) Virtuelles Gerät starten:

- Finden Sie Ihr neu erstelltes virtuelles Gerät in der Liste der verfügbaren AVDs im AVD Manager.
- Klicken Sie auf das grüne Wiedergabe-Symbol (Launch this AVD in the emulator), um das virtuelle Gerät zu starten.

7) Anwendung auf dem Emulator ausführen:

- Öffnen Sie Ihr Projekt in Android Studio.
- Wählen Sie das neu gestartete virtuelle Gerät aus der Dropdown-Liste in der Symbolleiste aus (neben dem grünen Run-Button).
- Klicken Sie auf den Run-Button (grünes Dreieck), um Ihre Anwendung auf dem Emulator auszuführen.

1) *Frontend:* Die Benutzeroberfläche der Pulsoximeter-App besteht aus mehreren Seiten, die verschiedene Funktionen und Informationen darstellen.

Hauptseite

Der Hauptbildschirm zeigt die aktuellen Messwerte des Pulsoximeters sowie Steuerungselemente für die Messung.

Hauptverantwortlichkeiten:

- **Titel TextView:** Zeigt den Titel "Pulsoximeter" auf dem Bildschirm an. Der Titel hat eine Textgröße von 24sp und ist horizontal zentriert mit einem oberen Randabstand von 40dp.
- **Anzeigebereich: SpO2 und Herzfrequenz:** Enthält zwei vertikale Layouts für die Anzeige von SpO2 (Sauerstoffsättigung) und Herzfrequenz. Der Anzeigebereich ist horizontal zentriert unterhalb des Titels mit einem oberen Randabstand von 20dp.
 - **SpO2-Anzeige:** Zeigt den Text "SpO2" und die aktuelle SpO2-Messung (Standardanzeige: "-"). Die Textfarbe ist dunkelblau und die Textgröße beträgt 24sp.
 - **Herzfrequenz-Anzeige:** Zeigt den Text "Herzfrequenz" und die aktuelle Herzfrequenz-Messung (Standardanzeige: "-"). Die Textfarbe ist dunkelblau und die Textgröße beträgt 24sp.
- **Liniendiagramm:** Zeigt ein Liniendiagramm der Herzfrequenzdaten an. Das Diagramm füllt die Breite des Bildschirms aus und befindet sich unterhalb des Anzeigebereichs und oberhalb der Steuerelemente mit Randab-

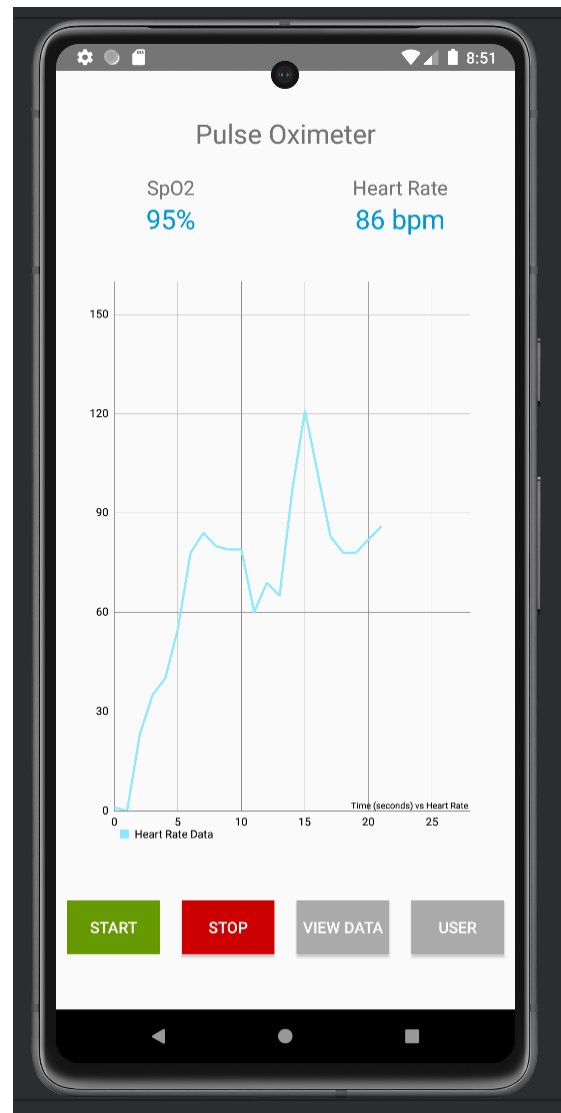


Abbildung 12. Datenanzeigesite

ständen von 25dp oben, links und rechts.

- **Steuerungsbereich:** Enthält Schaltflächen zum Starten, Stoppen, Anzeigen von Daten und Benutzereinstellungen. Der Steuerungsbereich ist horizontal zentriert am unteren Rand des Bildschirms mit oberen und unteren Randabständen von 40dp.
 - **Start-Button:** Schaltfläche zum Starten der Messung. Der Button ist grün mit weißem Text und hat eine Layout-Marge von 10dp.
 - **Stop-Button:** Schaltfläche zum Stoppen der Messung. Der Button ist rot mit weißem Text, hat eine Layout-Marge von 10dp und ist standardmäßig deaktiviert.
 - **Daten anzeigen-Button:** Schaltfläche zum Anzeigen der gespeicherten Daten. Der Button ist grau mit weißem Text und hat eine Layout-Marge von 10dp.

- **Benutzereinstellungen-Button:** Schaltfläche für die Benutzereinstellungen. Der Button ist grau mit weißem Text und hat eine Layout-Marge von 10dp.

Benutzereinstellungenseite

Dieser Bildschirm ermöglicht es dem Benutzer, seinen Benutzernamen einzugeben und zu speichern.

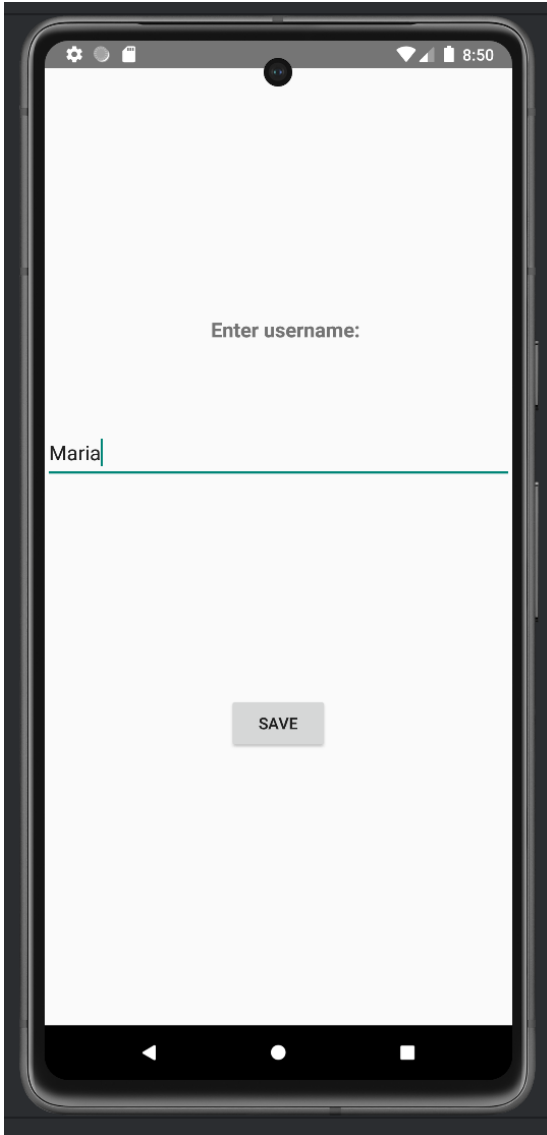


Abbildung 13. Benutzereinstellungenseite

Hauptverantwortlichkeiten:

- **Benutzereingabe TextView:** Zeigt den Text 'Enter username' zentriert auf dem Bildschirm an. Der Text dient als Aufforderung an den Benutzer, seinen Benutzernamen einzugeben.
- **Benutzereingabe EditText:** Ein Eingabefeld, in das der Benutzer seinen Benutzernamen eingeben kann. Das Eingabefeld ist direkt unter dem TextView positioniert.

- **Speichern-Button:** Eine Schaltfläche zum Speichern des eingegebenen Benutzernamens. Der Button ist grau mit weißem Text und ist unterhalb des Eingabefeldes zentriert platziert.

Datenanzeigeseite

Dieser Bildschirm zeigt die gespeicherten Messwerte der Benutzer an.

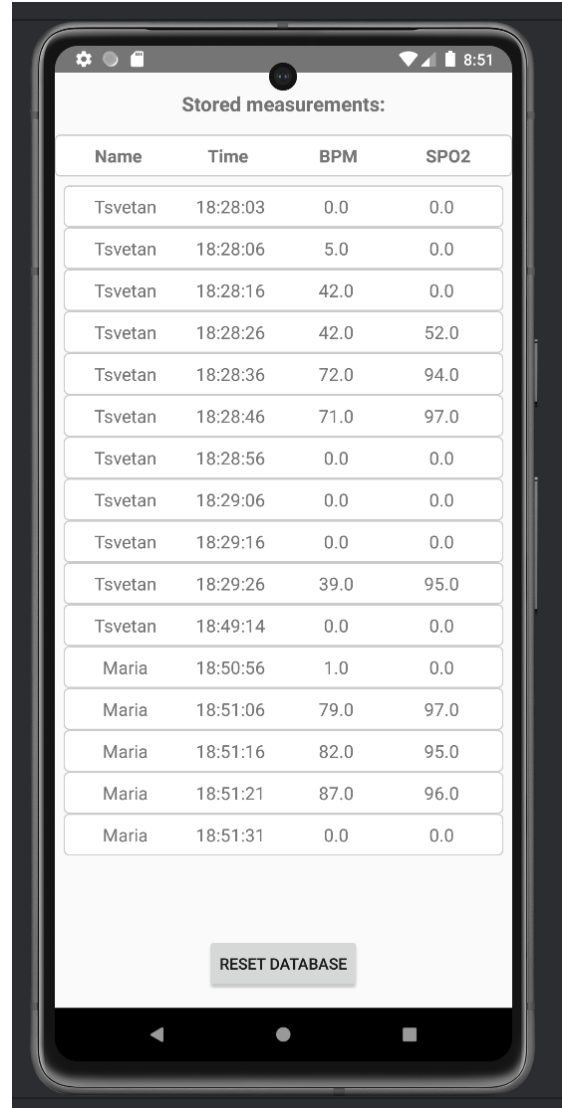


Abbildung 14. Datenanzeigeseite

Hauptverantwortlichkeiten:

- **Titel TextView:** Zeigt den Titel 'Stored measurements' zentriert auf dem Bildschirm an. Der Titel gibt an, dass auf diesem Bildschirm die gespeicherten Messwerte angezeigt werden.
- **Datenanzeige-Liste:** Eine Liste, die die gespeicherten Messungen darstellt. Jede Zeile der Liste enthält den Namen des Benutzers, die Uhrzeit der Messung, den

BPM (Herzfrequenz) und die SpO2-Werte. Die Liste ist so formatiert, dass die Daten klar und lesbar sind.

- **Datenbank zurücksetzen-Button:** Eine Schaltfläche zum Zurücksetzen der Datenbank. Der Button ist grau mit weißem Text und ist am unteren Rand des Bildschirms zentriert platziert. Durch das Drücken dieser Schaltfläche werden alle gespeicherten Daten gelöscht.

2) *Backend und Architekturüberblick:* Dieser UML-Diagramm gibt einen Überblick über unsere Backend-Architektur.

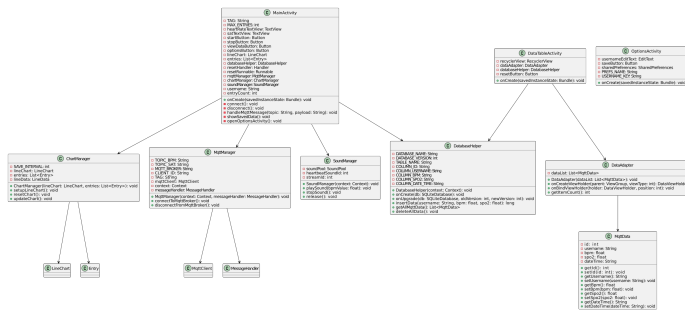


Abbildung 15. UML-Klassendiagramm

MainActivity

MainActivity ist die zentrale Komponente der Anwendung. Sie verwaltet die Hauptbenutzeroberfläche, einschließlich Buttons und Textansichten, und interagiert mit anderen Komponenten, um wichtige Aufgaben auszuführen.

Hauptverantwortlichkeiten:

- **Benutzeroberfläche:** Zeigt Herzfrequenz- und Sättigungswerte mithilfe von `heartRateTextView` und `satTextView` an. Außerdem bietet sie Buttons zum Starten und Stoppen der Datenerfassung, zum Anzeigen gespeicherter Daten und zum Aufrufen von Optionen.
- **Datenverwaltung:** Verwaltet eine Liste von Datenpunkten (entries) und aktualisiert das Diagramm (`lineChart`) mit neuen Daten.
- **MQTT-Verbindung:** Verwendet `MqttManager`, um eine Verbindung zu einem MQTT-Broker herzustellen, Nachrichten zu empfangen und diese angemessen zu verarbeiten.
- **Datenbankoperationen:** Nutzt `DatabaseHelper`, um Daten in der lokalen Datenbank zu speichern und abzurufen.
- **Soundmanagement:** Verwendet `SoundManager`, um basierend auf den Herzfrequenzdaten Töne abzuspielen.

ChartManager

ChartManager ist verantwortlich für die Verwaltung der Anzeige von Daten in einem `LineChart`.

Hauptverantwortlichkeiten:

- **Einrichten und Aktualisieren:** Initialisiert das Liniendiagramm und aktualisiert es mit neuen Datenpunkten.
- **Datenverwaltung:** Verwaltet eine Liste von `Entry`-Objekten und aktualisiert die `LineData` für das Diagramm.

DataAdapter

`DataAdapter` wird verwendet, um eine Liste von `MqttData`-Objekten in einem `RecyclerView` zu verwalten und anzuzeigen.

Hauptverantwortlichkeiten:

- **RecyclerView-Verwaltung:** Erstellt und bindet `View-Holders`, um die Datenpunkte anzuzeigen.
- **Datenbindung:** Bindet `MqttData`-Elemente an die Ansichten im `RecyclerView`.

DatabaseHelper

`DatabaseHelper` verwaltet die Erstellung und Verwaltung der lokalen Datenbank.

Hauptverantwortlichkeiten:

- **Datenbankoperationen:** Erstellt und aktualisiert die Datenbank. Außerdem bietet sie Methoden zum Einfügen neuer Daten, Abrufen aller Daten und Löschen aller Daten aus der Datenbank.

DataTableActivity

`DataTableActivity` zeigt die gesammelten Daten in tabellarischer Form mithilfe eines `RecyclerView` an.

Hauptverantwortlichkeiten:

- **UI-Verwaltung:** Initialisiert den `RecyclerView` und richtet den `DataAdapter` ein, um Daten anzuzeigen.
- **Datenbankinteraktion:** Verwendet `DatabaseHelper`, um Daten aus der Datenbank abzurufen und anzuzeigen.

MqttData

`MqttData` ist eine Datenklasse, die einzelne Datenpunkte repräsentiert, die vom MQTT-Broker gesammelt wurden.

Hauptverantwortlichkeiten:

- **Datenrepräsentation:** Enthält Attribute wie ID, Benutzername, Herzfrequenz (bpm), Sauerstoffsättigung (spo2) und Zeitstempel (dateTime). Bietet Getter- und Setter-Methoden für diese Attribute.

MqttManager

`MqttManager` verwaltet die Verbindung zum MQTT-Broker und verarbeitet eingehende Nachrichten.

Hauptverantwortlichkeiten:

- **Verbindungsmanagement:** Stellt eine Verbindung zum MQTT-Broker her und trennt diese wieder.
- **Nachrichtenverarbeitung:** Verarbeitet eingehende MQTT-Nachrichten und übergibt die Daten an MainActivity zur weiteren Verarbeitung.

OptionsActivity

OptionsActivity ermöglicht es Benutzern, Anwendungseinstellungen wie den Benutzernamen zu konfigurieren.

Hauptverantwortlichkeiten:

- **Einstellungsmanagement:** Bietet eine Benutzeroberfläche zur Eingabe und Speicherung des Benutzernamens mithilfe von `usernameEditText` und `saveButton`. Verwendet `SharedPreferences`, um diese Einstellungen zu speichern und abzurufen.

SoundManager

SoundManager verwaltet die Wiedergabe von Tönen basierend auf den Herzfrequenzdaten.

Hauptverantwortlichkeiten:

- **Soundwiedergabe:** Spielt Töne ab, stoppt und gibt Ressourcen mithilfe eines `SoundPool` frei. Spielt einen Herzschlagton basierend auf dem aktuellen Herzfrequenzwert ab.

Wie alles zusammenarbeitet

- **Start:** Beim Start der App initialisiert MainActivity die Benutzeroberfläche und richtet Verbindungen zu verschiedenen Managern ein (`ChartManager`, `MqttManager`, `SoundManager` usw.).
- **Datenerfassung:** Beim Start der Datenerfassung stellt `MqttManager` eine Verbindung zum MQTT-Broker her und abonniert Themen für Herzfrequenz- und Sauerstoffsättigungsdaten.
- **Datenempfang:** Wenn Daten vom MQTT-Broker empfangen werden, übergibt `MqttManager` diese an MainActivity, die die Benutzeroberfläche aktualisiert und die Daten mithilfe von `DatabaseHelper` speichert.
- **Datenanzeige:** MainActivity aktualisiert das `LineChart` über `ChartManager`, um Echtzeitdaten anzuzeigen. Benutzer können detaillierte Daten in tabellarischer Form über `DataTableActivity` anzeigen.
- **Audiowiedergabe:** Basierend auf den Herzfrequenzdaten spielt `SoundManager` einen entsprechenden Ton ab, um akustisches Feedback zu geben.

- **Benutzereinstellungen:** Benutzer können Einstellungen wie ihren Benutzernamen in `OptionsActivity` konfigurieren, die mithilfe von `SharedPreferences` gespeichert werden.

V. PROBLEME UND DISKUSSION

Das erste Problem, das wir hatten, war mit dem Ton. Am Anfang des Dateiladens hört man ein Rauschen. Das könnte entweder an der Qualität der Datei liegen, die wir verwenden, oder am Ladevorgang selbst. Ein weiteres Problem ist, dass beim Neuladen des Diagramms in der Anwendung tatsächlich eine kleine Verzögerung auftritt. Aus diesem Grund geht manchmal eine der Messungen verloren und wird nicht korrekt visualisiert.

VI. LITERATUR

LITERATUR

- [1] Tsvetan Stanchev Gruppe 20: Maria Lyoteva. *Pulse-Oximeter: Modularbeit für Mobile und Ubiquitous Computing*. Techn. Ber. 2024. Ostbayerische Technische Hochschule Amberg-Weiden, Studiengang: Künstliche Intelligenz, Juli 2024.
- [2] Leela Prasad. *Creating LineChart using MpAndroidChart*. Medium. Dec. 3, 2018. URL: <https://medium.com/@leelaprasad4648/creating-linechart-using-mpandroidchart-33632324886d> (visited on 07/09/2024).
- [3] *MAX30100 Pulse Oximeter Interfacing with ESP32 | ESP32*. URL: <https://www.electronicwings.com/esp32/max30100-pulse-oximeter-interfacing-with-esp32> (visited on 07/09/2024).
- [4] sam.moosavian. *Pulse Oximeter With ESP32*. Instructables. URL: <https://www.instructables.com/Heart-Rate-and-Blood-Oxygen-Level-With-Arduino-IDE/> (visited on 07/09/2024).
- [5] Arnov Sharma. *Oximeter With MAX30100 Heart Rate Sensor and ESP32*. Instructables. URL: <https://www.instructables.com/Oximeter-With-MAX30100-Heart-Rate-Sensor-and-ESP32/> (visited on 07/09/2024).
- [6] *Running Node-RED locally : Node-RED*. URL: <https://nodered.org/docs/getting-started/local> (besucht am 09.07.2024).
- [7] *Flows : Node-RED*. URL: <https://nodered.org/docs/user-guide/editor/workspace/flows> (besucht am 09.07.2024).
- [8] *node-red-dashboard*. URL: <http://flows.nodered.org/node/node-red-dashboard> (visited on 07/09/2024).
- [9] *Part 3: Installing and Using MQTT Broker (Mosquitto) & Node-Red*. HackMD. URL: https://hackmd.io/@lnu-iot/rJr_nGyq5 (visited on 07/09/2024).
- [10] *Connect to an MQTT Broker : Node-RED*. URL: <https://cookbook.nodered.org/mqtt/connect-to-broker> (besucht am 09.07.2024).
- [11] *Running on Windows : Node-RED*. URL: <https://nodered.org/docs/getting-started/windows> (besucht am 09.07.2024).
- [12] *AnyChart Android Charts | AnyChart*. URL: <https://www.anychart.com/technical-integrations/samples/android-charts/> (besucht am 09.07.2024).
- [13] *PhilJay/MPAndroidChart: A powerful Android chart view / graph view library, supporting line- bar- pie- radar- bubble- and candlestick charts as well as scaling, panning and animations*. URL: <https://github.com/PhilJay/MPAndroidChart> (besucht am 09.07.2024).
- [14] *AnyChart/AnyChart-Android: AnyChart Android Chart is an amazing data visualization library for easily creating interactive charts in Android apps. It runs on API 19+ (Android 4.4) and features dozens of built-in chart types*. URL: <https://github.com/AnyChart/AnyChart-Android> (besucht am 09.07.2024).
- [15] *Create and manage virtual devices | Android Studio | Android Developers*. URL: <https://developer.android.com/studio/run/managing-avds> (besucht am 09.07.2024).
- [16] Kelsey Nocek. *Setting Up an Android Emulator in Android Studio*. Medium. Sept. 25, 2023. URL: <https://medium.com/@kelseynocck12/setting-up-an-android-emulator-in-android-studio-3bc4542e3b3e> (visited on 07/09/2024).
- [17] Serhii Orlivskiy. *Mosquitto MQTT Broker on Windows Installation - Ultimate Guide*. Cedalo. Mar. 24, 2023. URL: <https://cedalo.com/blog/how-to-install-mosquitto-mqtt-broker-on-windows/> (visited on 07/09/2024).
- [18] EMQ Technologies Inc. *MQTTX: Your All-in-one MQTT Client Toolbox*. mqttx.app. URL: <https://mqttx.app/> (visited on 07/09/2024).
- [19] EMQ Technologies Inc. *Introduction*. mqttx.app. URL: <https://mqttx.app/> (visited on 07/09/2024).
- [20] *Interfacing MAX30100 Pulse Oximeter and Heart Rate Sensor with Arduino*. Last Minute Engineers. Feb. 5, 2022. URL: <https://lastminuteengineers.com/max30100-pulse-oximeter-heart-rate-sensor-arduino-tutorial/> (visited on 07/09/2024).
- [21] Matteo Trovò. *Guide to mastering MQTT on Android using Mosquitto*. Cedalo. Mar. 11, 2024. URL: <https://cedalo.com/blog/mqtt-on-android-guide-using-mosquitto/> (visited on 07/09/2024).
- [22] *Android and MQTT: A Simple Guide. How to develop an MQTT Client with...* | by Leonardo Cavagnis | The Startup | Medium. URL: <https://medium.com/swlh/android-and-mqtt-a-simple-guide-cb0cbb1931c> (besucht am 09.07.2024).
- [23] Mashruf Zaman. *Adding and Modifying an Android Library in Android Studio*. Medium. Jan. 13, 2024. URL: <https://medium.com/@mashruf.zaman/adding-and-modifying-an-android-library-in-android-studio-c6ab3cfd838e> (visited on 07/09/2024).
- [24] *How to Use a SQLite Database with Android Studio*. URL: <https://www.freecodecamp.org/news/how-to-use-sqlite-database-with-android-studio/> (besucht am 09.07.2024).
- [25] *How to Create and Add Data to SQLite Database in Android? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/how-to-create-and-add-data-to-sqlite-database-in-android/> (besucht am 09.07.2024).
- [26] *How to Create and Add Data to SQLite Database in Android? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/how-to-create-and-add-data-to-sqlite-database-in-android/> (besucht am 09.07.2024).
- [27] *How to Create and Add Data to SQLite Database in Android? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/how-to-create-and-add-data-to-sqlite-database-in-android/> (besucht am 09.07.2024).
- [28] *How to Create and Add Data to SQLite Database in Android? GeeksforGeeks*. Section: Android. Feb. 22, 2021. URL: <https://www.geeksforgeeks.org/how-to-create-and-add-data-to-sqlite-database-in-android/> (besucht am 09.07.2024).
- [29] *How to Read Data from SQLite Database in Android? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/how-to-read-data-from-sqlite-database-in-android/> (besucht am 09.07.2024).
- [30] *How to Read Data from SQLite Database in Android? GeeksforGeeks*. Section: Android. Feb. 22, 2021. URL: <https://www.geeksforgeeks.org/how-to-read-data-from-sqlite-database-in-android/> (visited on 07/09/2024).
- [31] *How to Update Data to SQLite Database in Android? GeeksforGeeks*. Section: Android. Feb. 22, 2021. URL: <https://www.geeksforgeeks.org/how-to-update-data-to-sqlite-database-in-android/> (visited on 07/09/2024).
- [32] *How to Delete Data in SQLite Database in Android? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/how-to-delete-data-in-sqlite-database-in-android/> (besucht am 09.07.2024).
- [33] Jonno_FTW. *Android: Heart Rate BPM sound playing*. Stack Overflow. 1. Sep. 2014. URL: <https://stackoverflow.com/q/25601384> (besucht am 09.07.2024).
- [34] marko. *Answer to Android: Heart Rate BPM sound playing*. Stack Overflow. 2. Sep. 2014. URL: <https://stackoverflow.com/a/25624894> (besucht am 09.07.2024).
- [35] *RestingHeartRateRecord*. Android Developers. URL: <https://developer.android.com/reference/androidx/health/connect/client/records/RestingHeartRateRecord> (visited on 07/09/2024).
- [36] *Java - Timer | Baeldung*. Nov. 2, 2014. URL: <https://www.baeldung.com/java-timer-and-timertask> (visited on 07/09/2024).